07 Articulo investigacion SaborExpress EN

SABOREXPRESS — PROYECTO DE SOFTWARE

Yerson Stiven Cuellar Rubiano

Ficha de formación: 3145555

Expediente técnico y académico — APA 7

14 de octubre de 2026

Fuente documental: repositorios SaborExpressBack, SaborExpressWeb y SaborExpressMovil.

Las pruebas no ejecutadas se identifican como pendientes.

Architecture and Traceability of a Web and Mobile Platform for Restaurant Operations: A Documentary Case Study of SaborExpress

Proyecto: SaborExpress | **Autor / integrante:** Yerson Stiven Cuellar Rubiano | **Ficha:** 3145555 | **Fecha:** 14 de octubre de 2026 | **Versión documental:** 4.0

Estado de la evidencia: inspección documental y estática del código. Las pruebas de ejecución y los datos institucionales no aportados no se presumen realizados.

Authors: [Confirm actual authors] | **Affiliation:** [Institution and academic programme] | **Corresponding author:** [To be confirmed]

Abstract

Objective: To characterize the source-evidenced architecture and validation gaps of SaborExpress, a restaurant-oriented information system combining web, mobile and backend components. **Method:** Descriptive and non-experimental documentary case study, grounded in static inspection of three public repositories and a targeted academic literature review on online food delivery, software architecture, security, digital identity and quality. **Results:** The backend targets ASP.NET Core 8 and includes Entity Framework Core, JWT, SignalR, business controllers and Wompi payment interfaces; the web project specifies Next.js 16 and React 19, while the mobile project uses Expo 57 and React Native 0.86. The inspected subset contains 52 declared HTTP methods across six controllers. **Conclusion:** Source inspection provides a traceability baseline, not evidence of system performance, user adoption, security compliance or successful real-world transactions.

Keywords: online food delivery; software engineering; restaurant management; web and mobile integration; traceability; digital payments.

1. Introduction

Online food delivery connects customers, kitchen operations, payment systems and delivery agents. Shroff et al. (2022) report a structured review of 43 studies and identify several research themes in food delivery. A complementary synthesis in the International Journal of Hospitality

Management (2022) discusses theories and methods used in this field. Such evidence motivates analysing both technology and operational processes; however, the prior results are not direct evidence of SaborExpress outcomes. The system offers a concrete software artefact for investigating code structure and the limits of repository-based evidence.

2. Research problem and questions

Restaurant processes require coordination across products, locations, orders, payments and delivery. Potential inconsistencies can emerge when external payment notifications and internal order states are processed asynchronously, or when a user accesses resources assigned to another branch. Without an operational baseline, the study cannot quantify the prevalence or economic consequences of these risks. The central question is: **What integration capabilities and verification gaps are visible from the SaborExpress codebase?** Subquestions address technology structure, domain relationships, security-sensitive interfaces and future quality evaluation.

Research questions and operational definitions

The primary question guiding this case study is: what documentary evidence does the SaborExpress codebase provide about integrating restaurant ordering, payment processing, delivery management, and multiple clients through a central API? Four supporting questions address (RQ1) functionality observable in source files, (RQ2) architecture and technology decisions, (RQ3) safety and consistency risks at the boundaries between orders, payments, deliveries, and identity, and (RQ4) additional evidence needed before any claim of operational readiness.

The unit of analysis is a software repository, with controllers, models, configuration artifacts and client packages as embedded units. The study does not formulate statistical causal hypotheses, recruit participants or estimate economic savings. Source-code observation can support architectural descriptions; it cannot directly demonstrate live reliability, customer satisfaction, or service performance. Distinguishing those levels of inference is a central methodological safeguard.

3. Related work

Food delivery research includes service reliability, convenience, perceived trust and logistics. The literature on delivery application service attributes (2021; DOI 10.1016/j.ijgfs.2021.100392), service quality and loyalty (British Food Journal, 2022) and customer satisfaction in a pandemic context (Prasetyo et al., 2021) indicates constructs that may be evaluated. The 2026 Colombian study by García-Umaña et al. relates continuance intention to an extended UTAUT2 model, delivery trust and perceived food quality, which provides locally relevant background without replacing project-specific field data. Davis (1989) and Venkatesh et al. (2003) underpin acceptance measures; Nielsen's heuristics can inform interface audits. Kruchten (1995) guides multiple views of architecture, while Taibi et al. (2018) provide a contrast with microservice patterns. ISO/IEC/IEEE 29148:2018, ISO/IEC 25010:2023, OWASP API Security Top 10 (2023) and NIST SP 800-63B-4 (2025) frame requirements, quality and security. None of these references certifies the analysed system.

Restaurant platforms and user trust

Online food delivery research treats the service as a multi-stakeholder system. Product discoverability, payment transparency, fulfillment quality and trust are intertwined with application design. Shroff, Shah and Gajjar (2022) reviewed 43 studies across 24 journals, identifying research themes that motivate scrutiny of the interaction between customers, operations and delivery personnel. Their findings justify examining a restaurant platform at both software and workflow levels without implying performance outcomes for the present case.

The Technology Acceptance Model (Davis, 1989) emphasizes perceived usefulness and perceived ease of use. Unified Theory of Acceptance and Use of Technology (Venkatesh et al., 2003) provides another framework for future analysis of adoption. For SaborExpress, such constructs could be operationalized through tasks such as placing an order, understanding payment confirmation and locating a delivery. However, the current investigation has not collected participant responses; therefore it does not report TAM or UTAUT scores.

Research by Prasetyo and colleagues (2021) studied factors affecting customer satisfaction and loyalty under pandemic conditions, in a setting distinct from the restaurant represented by the repositories. Its findings cannot simply be transferred to Colombia or to different operational roles. Nielsen's usability heuristics, last reviewed in 2024, are similarly useful as evaluation criteria rather than as evidence of the interface's current usability.

4. Methodology

The research employs a documentary, descriptive, cross-sectional case study inspired by Yin (2018). Units of analysis are yersonct/SaborExpressBack, yersonct/SaborExpressWeb and yersonct/SaborExpressMovil, together with the documentation repository, observed in October 2026. Inspection prioritised Program.cs, project manifests, data configuration, key controllers and entity models. Each claim was classified as directly evidenced, interpretative or pending operational validation. The literature review was targeted, not a formally registered systematic review; no PRISMA selection statistics are claimed. No participants, database production records, live payments or performance benchmarks were collected. Replication requires pinning exact commit SHAs.

Research protocol and evidence hierarchy

A reproducible analysis starts by recording repository name, branch and commit identifier. Dependency manifests establish declared technology versions, while bootstrap files describe service registration and middleware composition. Domain models and API controllers are then inspected to recover public operations, relationships, policy attributes and integration points. Each statement is linked to a concrete path and labeled as directly observed, inferred from structural elements, or recommended for future work.

The research uses a directed literature review. Relevant publications on food delivery, information-system acceptance, software architecture and API security provide analytical dimensions rather than empirical outcomes for the evaluated application. Reports by Shroff et al. (2022) and Prasetyo et al. (2021) address broader service contexts; theoretical work by Davis (1989) and Venkatesh et al. (2003) frames potential user studies. No PRISMA-style systematic

review or quantitative meta-analysis was conducted, and the study must not be represented as either.

The evidence matrix records source path, observed element, interpretation, associated risk, confidence level and future verification activity. A controller method provides evidence of an implemented entry point, not proof that it is available to every relevant actor or that its operation is reliable. A package dependency reveals declared tooling, not deployment success. An integration endpoint receiving signed payment events does not by itself prove correct cryptographic verification. These distinctions prevent overstated conclusions.

5. Results

5.1. Source architecture

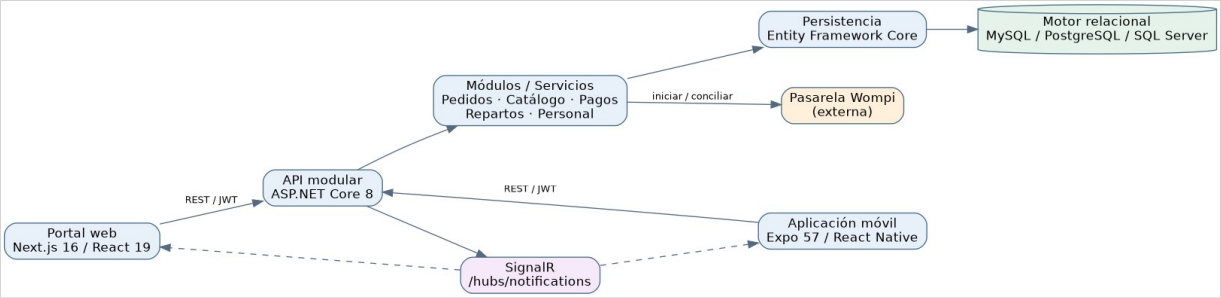
The backend SaborExpress.csproj specifies net8.0; Program.cs registers dependency injection, controllers, authentication/authorization, Swagger, SignalR, background services, migrations and seeders. DatabaseConfiguration.cs selects MySQL, SQL Server or PostgreSQL based on a string configuration value. The web manifest declares Next.js 16.2.6 and React 19.2.4. The mobile manifest lists Expo ~57.0.27 and React Native 0.86.3. These are declared dependencies, not proven production runtime versions.

5.2. Domain and interfaces

The analysed six controllers cover authentication, orders, products, payments, deliveries and branches. Their 52 annotated HTTP operations include session management, CRUD actions, status changes, Wompi payment initialisation and webhook handling, and delivery rating. Other controller modules are present but not all their routes were enumerated for this study. Order relates to a branch, optional customer/employee/table/address, details and payments. Payment carries Wompi reference fields; Product.BranchId is nullable for global items; Delivery includes assigned personnel, status and optional rating. These observations do not establish underlying SQL DDL correctness.

5.3. Security-sensitive design surfaces

Potential audit areas include development-oriented CORS handling, automatic database migration during startup, logging of webhook payloads, and authorisation semantics for branch-related paths. The webhook allows anonymous HTTP access as expected for an external provider but requires service-side validation of signature, reference and replay resistance. No penetration testing was performed, and the observations are not reported as exploited vulnerabilities.



Logical architecture derived from inspected code

Figure 1. Logical component model derived from source code, not verified deployment topology.

Technical results from inspected repositories

The backend targets .NET 8 and declares libraries for ASP.NET Core authentication, Entity Framework Core, Swagger and relational database providers. Program.cs registers controllers, the JWT configuration, services, real-time notifications through SignalR, background tasks and database migration/seed behavior. The static organization is consistent with a modular ASP.NET Core application. The reviewed material does not justify labeling the backend a production microservices architecture.

The web client declares Next.js 16.2.6 and React 19.2.4; the mobile client declares Expo 57 and React Native 0.86.3. Their separation across repositories may support independent client development, but creates a version-compatibility challenge when the central API changes payloads, enum values or authorization requirements. Contract snapshots and coordinated compatibility checks are therefore appropriate next steps.

The order domain includes an Order entity associated with a mandatory branch, optional customer, employee, table and address, order channel, total-related amounts, status and timestamps. Related OrderDetail records store product, quantity, unit price and subtotal. OrdersController exposes operations for creation, retrieval, updates, status changes and cancellation. These are strong documentary indicators of domain scope, but no transactional consistency or concurrency guarantee can be claimed without targeted tests.

PaymentsController identifies direct customer payment initiation, cashier QR initiation, a webhook, transaction synchronization, refund and shift queries. The Payment model contains order linkage, payment amount, method, status and optional Wompi reference and transaction identifiers. The webhook reads the raw payload and the X-Event-Checksum header before invoking an application service. Correct cryptographic verification, amount correlation and idempotent persistence remain questions for dynamic testing. The Wompi events documentation describes signature verification and the separation of sandbox and production environments.

Delivery management exposes assignment, listing by order or employee, status updates and customer ratings. Delivery references the order, delivery address and assigned employee, while some controller methods explicitly compare the current actor, role and branch. These observations motivate negative tests for unauthorized access to another employee's delivery, and for attempts to bypass branch restrictions. OWASP's 2023 API Security Top 10 identifies object-level authorization among key risk categories.

The backend's database configuration supports selection among MySQL, SQL Server and PostgreSQL by configuration; actual portability requires confirming the migrations and queries on each provider. Startup migration through MigrateAsync() also motivates deliberate release sequencing and rollback procedures. Static analysis cannot establish that any particular cloud, backup strategy or managed database is currently in production.

6. Discussion

The code reveals separation of interface and backend responsibilities and a central domain model for operational processes. A modular monolithic application must not be called a deployed microservices architecture merely because it contains independent modules. Prior food-delivery studies highlight trust, delivery experience and quality; these motivate evaluation criteria rather than providing evidence of local satisfaction. An API with JWT requires resource-scoped authorization testing, especially for cross-branch resources. The existence of three selectable database providers is a portability design affordance that remains to be verified with provider-specific migrations and integration tests. Similarly, the Wompi controller demonstrates an integration boundary but not confirmed real-world payment correctness.

Quality risks and architectural decisions

ISO/IEC 25010:2023 provides a product-quality model that helps frame evaluation activities. Functional suitability should be tested against end-to-end workflows; performance efficiency requires defined workload and latency measurements; reliability requires controlled failures; compatibility requires client and provider combinations; security requires both role and resource-level authorization; maintainability requires stable module boundaries and regression checks. None of these characteristics is certified merely because a framework supports a feature.

A notable integration risk is contractual drift between the central API and the independent web and mobile clients. Updating a payment state or order representation may lead to a broken screen if consumers are not tested against the corresponding server version. Versioned OpenAPI snapshots, build pipelines and a small cross-client smoke suite are practical, proportionate controls. These do not require immediate adoption of a distributed microservice architecture.

Real-time delivery through SignalR may improve perceived responsiveness, but should not be mistaken for durable event delivery. Mobile clients lose connectivity; therefore clients must reconcile current business state through authorized REST queries upon reconnection. Payment flows deserve even stronger invariants: a customer redirected back from a payment provider

should not be shown a settled payment until server-side confirmation has been authenticated and persisted. Retries must not cause a second transaction.

Authorization is also contextual. Checking a JWT or role claim only identifies the requester; it does not establish ownership of an order, branch or delivery. Every sensitive operation should compare the resource's actual associations with the authorized actor. Test fixtures involving two customers, two branches and multiple delivery employees can expose this category of defect without using actual personal data.

The architectural documentation follows Kruchten's principle of multiple views, distinguishing the logical organization, development structure, runtime processes and proposed deployment. A deployment diagram should be clearly labeled as proposed unless actual infrastructure configuration and runtime logs are available. Likewise, three database providers in project dependencies do not constitute an observed high-availability architecture.

7. Proposed evaluation plan

Area	Suggested test	Observable outcome	State
Functional	Order creation/editing and state transition	Integrity and role-permitted changes	Not run
Payments	Wompi sandbox approved/rejected/duplicate webhooks	No duplicated posting	Not run
Security	Cross-user and cross-branch attempts	Denials without data leaks	Not run
Realtime	Concurrent SignalR sessions and reconnect	Measured delivery/consistency	Not run
Performance	Controlled HTTP load with instrumented sampling	p50/p95 with workload	Not run
Usability	Consent-based task sessions and questionnaires	Task success, errors, perceived ease	Not run
Recovery	Backup/restore and migration rollback rehearsal	Documented recovery	Not run

Research should pre-register acceptable thresholds, environments, credentials policy, data anonymisation and analysis procedures. Proposed metrics are not presented as achieved performance.

Measurable follow-up protocol

Construct	Proposed measure	Method	Boundary
Financial consistency	duplicate settled payments per external event	sandbox replay and database verification	no live payments

Construct	Proposed measure	Method	Boundary
Authorization	successful unauthorized object requests	two-branch negative API suite	authorized test environment
Performance	p50/p95 order submission time	versioned synthetic load	no pre-claimed result
Reliability	successful recovery after disconnection	controlled client-network failures	isolated setup
Usability	task success and errors	consented role-based evaluation	not yet conducted
Portability	successful migrations per DB provider	clean ephemeral database	requires actual provider instances
Maintainability	contract regressions after DTO change	OpenAPI diff and cross-client suite	requires CI execution

8. Threats to validity

Construct validity is limited because source presence does not prove correct runtime behaviour. Internal validity is limited by unknown deployment settings and external integrations. External validity is limited by absence of real users or restaurant operational data. Reproducibility requires fixed commit identifiers and a known database engine. A targeted literature sample does not support exhaustiveness claims.

Transferability and future evaluation

Construct validity is limited by treating source-code structure as a proxy for system capability. The study mitigates this risk by using cautious wording and linking each claim to an inspected path, while leaving functional validation pending. Internal validity is constrained by the absence of executed end-to-end tests, user interviews, performance benchmarks and a running sandbox environment. External validity is narrow: the observations concern the documented SaborExpress repositories at a particular point in time, not all restaurant management applications.

A next-phase evaluation should version the repositories, record environment and test data, execute both positive and negative scenarios, and store anonymized evidence for each test. Important scenarios include invalid Wompi signatures, replayed webhook events, late cancellation requests, cross-branch access, concurrent updates, network interruption, schema migration and restoration from backup. Quantitative results must be reported only after workloads and measurement tools have been specified and used.

A user evaluation would need clearly defined participant recruitment, informed consent, task scripts, success criteria, time-on-task instrumentation and data-handling controls. Only then could acceptance models be operationalized and contrasted with prior literature. This case study does not claim any such evidence at present.

The design is documented using public abstracts and official material for ISO standards, which may be access restricted. The paper does not reproduce copyrighted normative text or claim ISO certification.

9. Conclusion

The reviewed repositories support a grounded description of a modular backend with separate web and mobile applications for restaurant operations, including order processing, external payment initiation, delivery coordination and role-specific access. They also reveal critical boundaries where architectural quality must be demonstrated rather than presumed.

The value of this case study lies in its traceability from concrete implementation artifacts to research questions, engineering controls and a falsifiable quality-assurance agenda. Declaring all functionalities complete or experimentally validated would exceed the available evidence. A defensible next stage is to extract versioned API and database contracts, execute security and consistency tests, collect controlled performance and user-experience observations, and update the analysis transparently with both successful and negative outcomes.

SaborExpress can be characterised as a multi-client restaurant platform with a modular ASP.NET Core API, EF Core persistence, JWT, SignalR and declared web and mobile clients. The most defensible outcome of the current study is architecture traceability and a testable validation agenda. Future work must observe actual payment integrity, cross-branch access controls, interface usability, recovery and performance before drawing causal conclusions about service improvements.

10. Ethical statement and declarations

No personal user records or financial credentials were used. The authors must provide their actual affiliation, funding, contribution and conflict-of-interest declarations before submission. The use of AI assistance, if applicable, must be disclosed according to the receiving institution's rules.

References

Davis, F. D. (1989). Perceived usefulness, perceived ease of use, and user acceptance of information technology. *MIS Quarterly*, *13*(3), 319–340. https://doi.org/10.2307/249008

Expo. (s. f.). *Expo documentation*. https://docs.expo.dev/

Fakfare, P. (2021). Influence of service attributes of food delivery application on customers' satisfaction and their behavioural responses: The IPMA approach. *International Journal of Gastronomy and Food Science*, *25*, 100392. https://doi.org/10.1016/j.ijgfs.2021.100392

Fielding, R., Nottingham, M., & Reschke, J. (2022). *HTTP semantics* (RFC 9110). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc9110

García-Umaña, A., Bernal-Peralta, J., Cevallos Uve, G. E., Alcívar Chávez, A. C., Córdoba, É. F., & Beserra, V. (2026). Continuance intention in online food delivery platforms in Colombia: Extending UTAUT2 with delivery trust and food quality perception. *Foods*, *15*(14), 2430. https://doi.org/10.3390/foods15142430

GitHub. (s. f.). *GitHub Actions documentation*. https://docs.github.com/en/actions

ISO/IEC. (2023). *ISO/IEC 25010:2023: Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model*. https://www.iso.org/standard/78176.html

ISO/IEC/IEEE. (2018). *ISO/IEC/IEEE 29148:2018: Systems and software engineering — Life cycle processes — Requirements engineering*. https://www.iso.org/standard/72089.html

Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc7519

Kitchenham, B., & Charters, S. (2007). *Guidelines for performing systematic literature reviews in software engineering* (EBSE-2007-01). https://homepages.dcc.ufmg.br/~figueiredo/disciplinas/papers/guidelines-kitchenham.pdf

Koay, K. Y., Cheah, C. W., & Chang, Y. X. (2022). A model of online food delivery service quality, customer satisfaction and customer loyalty: A combination of PLS-SEM and NCA approaches. *British Food Journal, 124*(12), 4516–4532. https://doi.org/10.1108/BFJ-10-2021-1169

Kruchten, P. (1995). The 4+1 view model of architecture. *IEEE Software, 12*(6), 42–50. https://doi.org/10.1109/52.469759

Microsoft. (s. f.). *ASP.NET Core documentation*. https://learn.microsoft.com/en-us/aspnet/core/

Microsoft. (s. f.). *Entity Framework Core documentation*. https://learn.microsoft.com/en-us/ef/core/

Microsoft. (s. f.). *SignalR documentation*. https://learn.microsoft.com/en-us/aspnet/core/signalr/introduction

Next.js. (s. f.). *Next.js documentation*. https://nextjs.org/docs

Nielsen, J. (2024). *10 usability heuristics for user interface design*. Nielsen Norman Group. https://www.nngroup.com/articles/ten-usability-heuristics/

OWASP Foundation. (2023). *OWASP API Security Top 10 — 2023*. https://owasp.org/API-Security/

Postman. (s. f.). *Collections overview*. https://learning.postman.com/docs/collections/collections-overview/

Prasetyo, Y. T., Tanto, H., Mariyanto, M., Hanjaya, C., Young, M. N., Persada, S. F., Miraja, B. A., & Redi, A. A. N. P. (2021). Factors affecting customer satisfaction and loyalty in online food delivery service during the COVID-19 pandemic: Its relation with open innovation. *Journal of Open Innovation: Technology, Market, and Complexity, 7*(1), 76. https://doi.org/10.3390/joitmc7010076

Project Management Institute. (2021). *A guide to the project management body of knowledge (PMBOK® Guide)* (7th ed.). https://www.pmi.org/pmbok-guide-standards/foundational/pmbok

Shankar, A., Jebarajakirthy, C., Nayal, P., Maseeh, H. I., Kumar, A., & Sivapalan, A. (2022). Online food delivery: A systematic synthesis of literature and a framework development. *International Journal of Hospitality Management, 104*, 103240. https://doi.org/10.1016/j.ijhm.2022.103240

Shroff, A., Shah, B. J., & Gajjar, H. (2022). Online food delivery research: A systematic literature review. *International Journal of Contemporary Hospitality Management, 34*(8), 2852–2883. https://doi.org/10.1108/IJCHM-10-2021-1273

Taibi, D., Lenarduzzi, V., & Pahl, C. (2018). Architectural patterns for microservices: A systematic mapping study. In *Proceedings of the 8th International Conference on Cloud Computing and Services Science* (pp. 221–232). https://doi.org/10.5220/0006798302210232

Temoshok, D., Choong, Y.-Y., Regenscheid, A., Galluzzo, R., Fenton, J., Richer, J., & Lefkovitz, N. (2025). *Digital identity guidelines: Authentication and authenticator management* (NIST SP 800-63B-4). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-63B-4

Venkatesh, V., Morris, M. G., Davis, G. B., & Davis, F. D. (2003). User acceptance of information technology: Toward a unified view. *MIS Quarterly, 27*(3), 425–478. https://doi.org/10.2307/30036540

Wompi. (s. f.). *Documentación para desarrolladores*. https://docs.wompi.co/

Yin, R. K. (2018). *Case study research and applications: Design and methods* (6th ed.). SAGE Publications. https://study.sagepub.com/yin6e

Appendix A. Architecture figures and traceable evidence

The following diagrams reconstruct architecture and domain relationships from the project repositories. They are illustrations of source-code inspection, not empirical evidence that the system has been deployed successfully or that business outcomes improved.

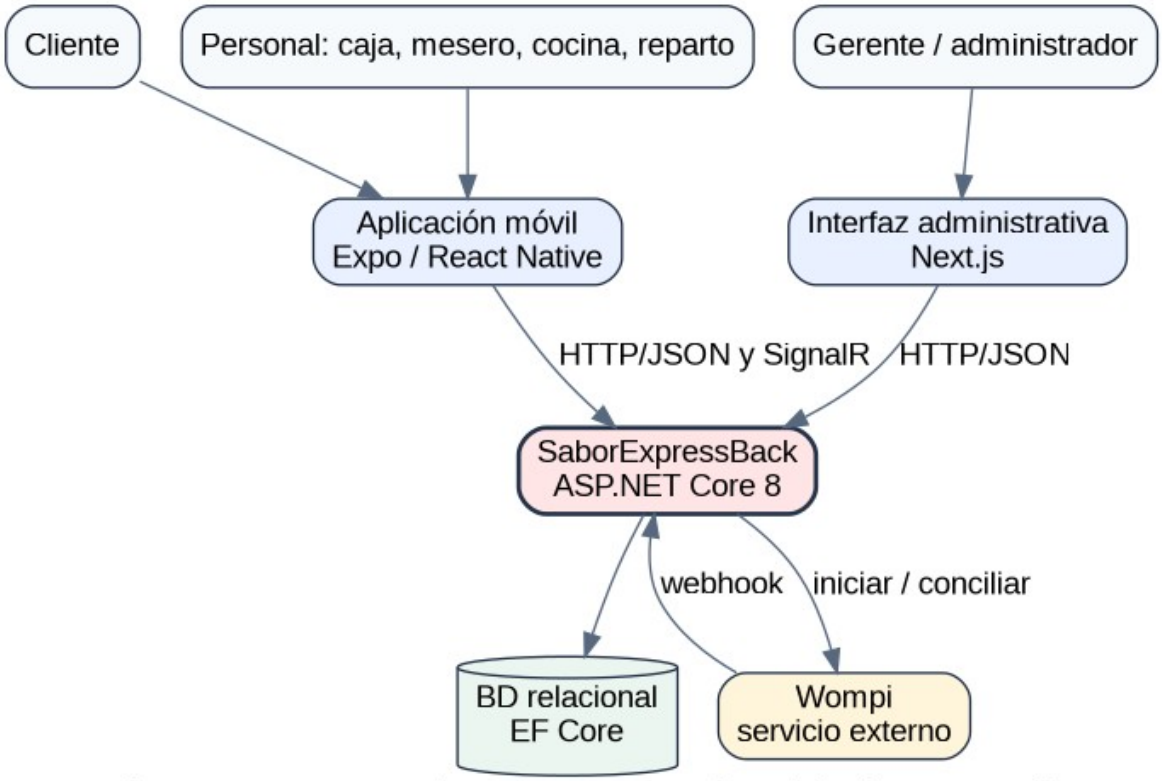


Figura 1. Contexto de SaborExpress (modelo documental)

01_CONTEXT0

Nota. Elaboración para el estudio de caso SaborExpress; las figuras no certifican operaciones desplegadas.

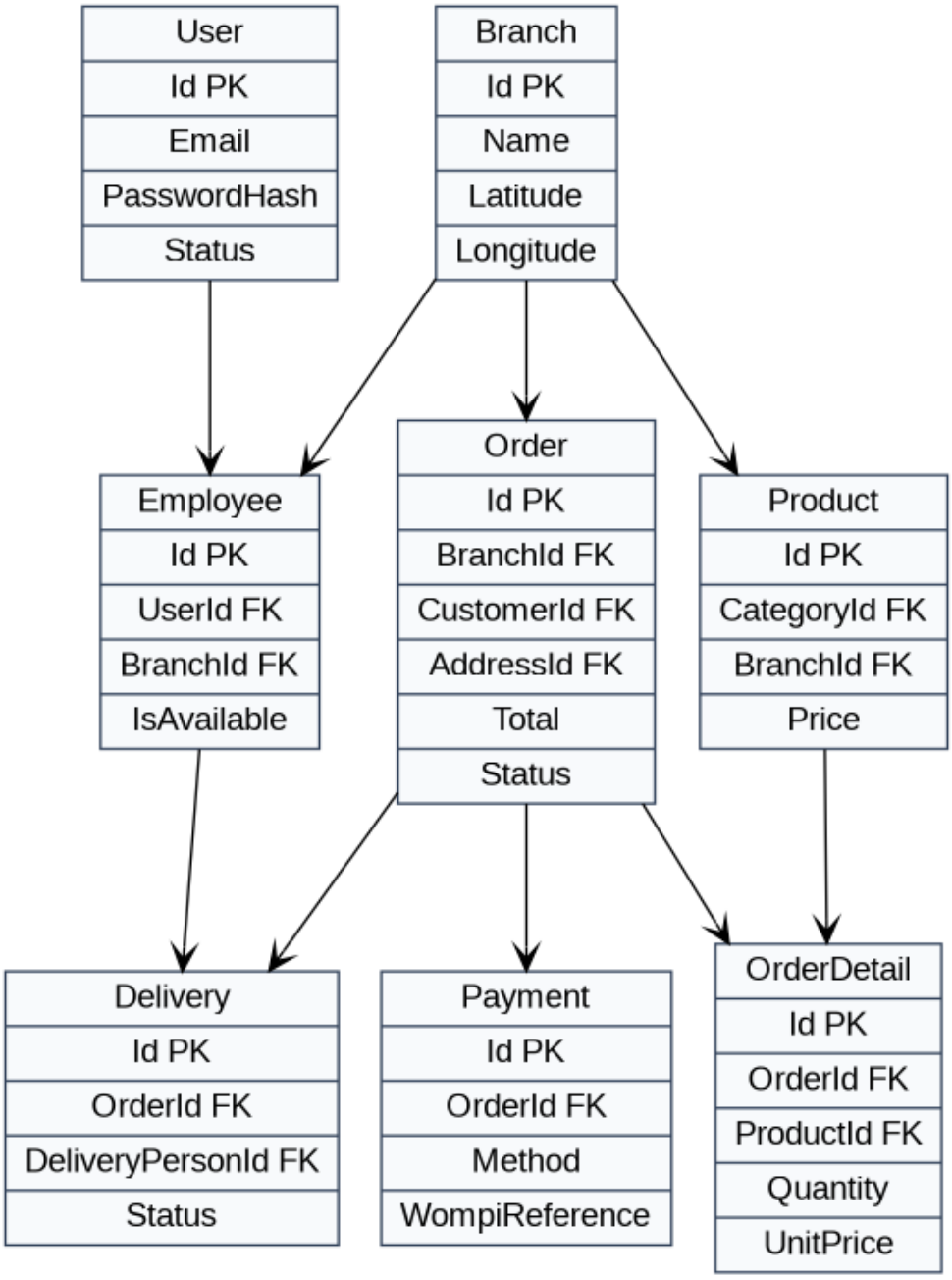


Figura 6. Modelo relacional conceptual extraído de modelos C#

06_MODELO_ER

Nota. Elaboración para el estudio de caso SaborExpress; las figuras no certifican operaciones desplegadas.

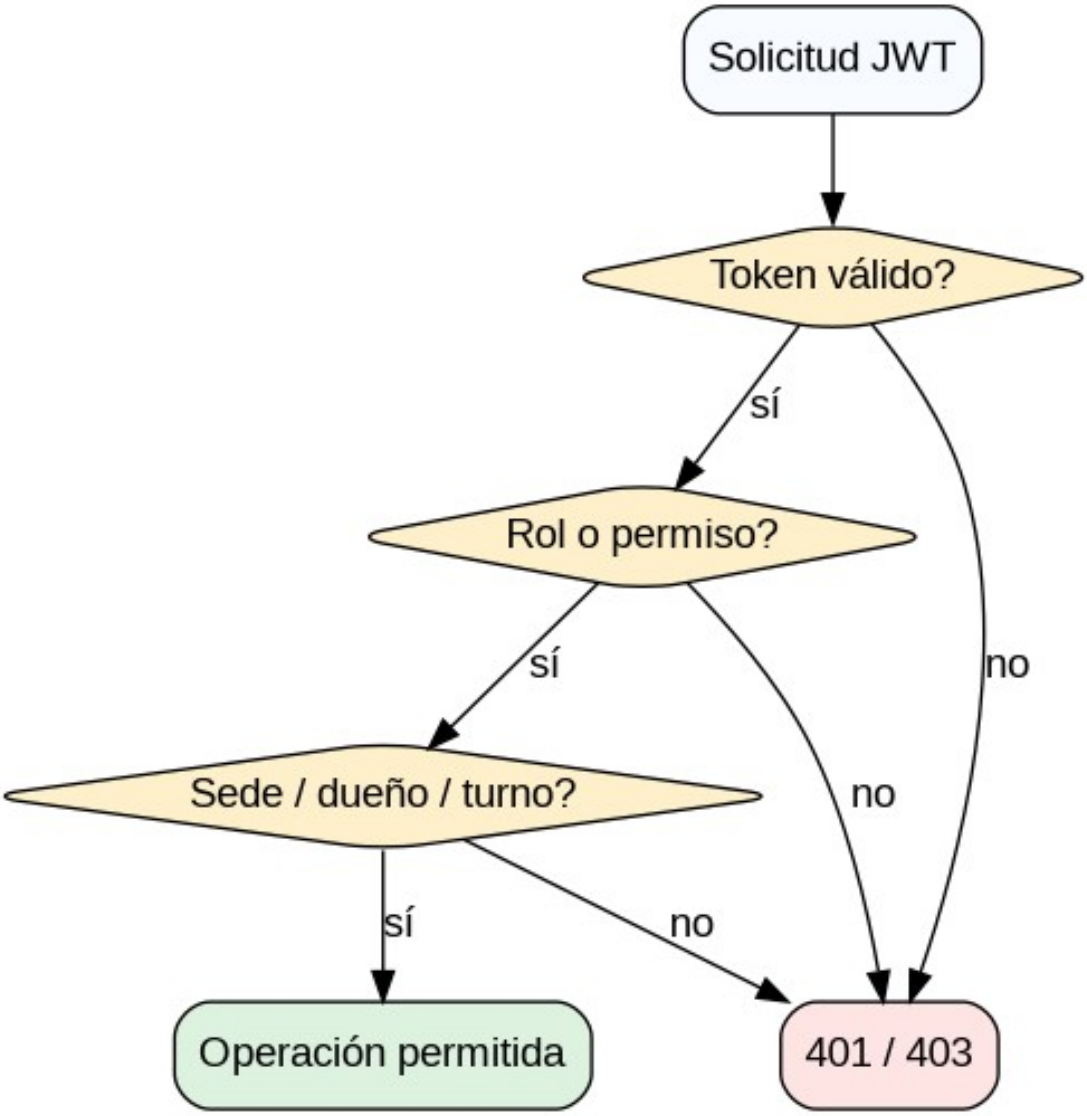


Figura 9. Controles de autorización que deben validarse

09_AISLAMIENTO_ROLES

Nota. Elaboración para el estudio de caso SaborExpress; las figuras no certifican operaciones desplegadas. ## Appendix B. Methodological limitations and future validation

Construct validity: an HTTP route documents an application entry point but does not prove complete business behavior. **Internal validity:** no before/after comparison, controlled experiment or field measurement was available, so causal claims about delivery times or

customer satisfaction are excluded. **External validity:** findings are limited to this software case and are not generalized to all restaurants. **Reliability:** future researchers can repeat the inspection by freezing repository commits and applying the same evidence classification scheme.

The proposed validation protocol calls for synthetic data, controlled Wompi sandbox transactions, security regression tests, load testing with documented virtual users and latency percentiles, and supervised usability observations with informed consent. Results should be reported only after such tests have actually been carried out.